rqbit 리소스 제한 및 QoS 구현 분석

개요

이 문서는 rqbit BitTorrent 클라이언트의 리소스 제한 메커니즘과 QoS(Quality of Service) 구현을 분석합니다. rqbit은 다층적인 리소스 관리 시스템을 통해 안정적인 성능과 시스템 보호를 제공합니다.

1. 동시 연결 수 및 메모리 사용량 제한

1.1 세마포어 기반 동시성 제어

rqbit은 세마포어를 사용하여 다양한 수준에서 동시성을 제어합니다:

```
pub  struct Session {
// 동시 토렌트 초기화 수 제한
pub(crate) concurrent_initialize_semaphore: Arc<tokio::sync::Semaphore>,
// ...
}

// 세마포어 초기화 (기본값: 3개 동시 초기화)
concurrent_initialize_semaphore: Arc::new(tokio::sync::Semaphore::new(
opts.concurrent_init_limit.unwrap_or(3),
)),
```

동시성 제어 특징:

- **토렌트 초기화**: 메모리 집약적인 초기화 작업의 동시 실행 수 제한
- **리소스 보호**: 과도한 메모리 사용으로 인한 시스템 불안정 방지
- **구성 가능**: 사용자 설정에 따른 동적 조정 가능

1.2 피어 연결 관리

```
// 피어 핸들러에서의 요청 세마포어
pub struct PeerHandler {
requests_sem: Semaphore, // 피어별 요청 수 제한
// ...
}

// 피어 연결 옵션
#[derive(Default, Debug, Copy, Clone, Serialize, Deserialize, Eq, PartialEq)]
pub struct PeerConnectionOptions {
pub connect_timeout: Option<Duration>,
pub read_write_timeout: Option<Duration>,
pub keep_alive_interval: Option<Duration>,
}
```

연결 관리 특징:

- **타임아웃 제어**: 연결, 읽기/쓰기 작업에 대한 시간 제한
- **Keep-alive**: 연결 유지를 위한 주기적 신호
- **피어별 제한**: 개별 피어에 대한 요청 수 제한

1.3 메모리 사용량 제한

```rust
// 디스크 쓰기 지연을 통한 메모리 사용량 제어
pub struct SessionOptions {
// 메모리에 버퍼링할 최대 크기 (MB 단위)
pub defer_writes_up_to: Option<usize>,
// ...
}

// 디스크 작업 큐 크기 계산
let (disk_write_tx, disk_write_rx) = opts
.defer_writes_up_to
.map(|mb| {
const DISK_WRITE_APPROX_WORK_ITEM_SIZE: usize = CHUNK_SIZE as usize + 300;
let count = mb * 1024 * 1024 / DISK_WRITE_APPROX_WORK_ITEM_SIZE;
let (tx, rx) = tokio::sync::mpsc::channel(count);
(Some(tx), Some(rx))
})
.unwrap_or((None, None));
```

메모리 제한 특징:

- **버퍼 크기 제한**: 메모리 사용량에 따른 디스크 쓰기 큐 크기 조정
- **백프레셔**: 메모리 부족 시 자동 백프레셔 적용

- **예측 가능한 사용량**: 청크 크기 기반 메모리 사용량 예측

2. 세마포어와 레이트 리미터 구현

2.1 고급 레이트 리미터 시스템

```
use arc_swap::ArcSwapOption;

use governor::DefaultDirectRateLimiter as RateLimiter;

use governor::Quota;


struct Limit(ArcSwapOption<RateLimiter>);


impl Limit {

fn new_inner(bps: Option<NonZeroU32>) -> Option<Arc<RateLimiter>> {

let bps = bps?;

Some(Arc::new(RateLimiter::direct(Quota::per_second(bps))))

}


async fn acquire(&self, size: NonZeroU32) -> crate::Result<()> {

let lim = self.0.load().clone();

if let Some(rl) = lim.as_ref() {

rl.until_n_ready(size).await?; // 토큰 버킷 알고리즘

}

Ok(())
```

```
}

fn set(&self, limit: Option<NonZeroU32>) {
let new = Self::new_inner(limit);
self.0.swap(new); // 원자적 교체
}
}
```

레이트 리미터 특징:

- **토큰 버킷**: Governor 크레이트의 토큰 버킷 알고리즘 사용
- **원자적 업데이트**: `ArcSwapOption` 을 통한 무잠금 설정 변경
- **비동기 대기**: 토큰 부족 시 비동기 대기로 백프레셔 구현

2.2 계층화된 대역폭 제어

```
pub struct Limits {
down: Limit, // 다운로드 제한
up: Limit, // 업로드 제한
}

impl Limits {
pub async fn prepare_for_upload(&self, len: NonZeroU32) -> crate::Result<()> {
self.up.acquire(len).await
}
```

```
pub async fn prepare_for_download(&self, len: NonZeroU32) -> crate::Result<()> {
self.down.acquire(len).await
}
}

// 업로드 스케줄러에서의 이중 제한 적용
async fn task_upload_scheduler(self: Arc<Self>, ...) -> crate::Result<()> {
while let Some((tx, ci)) = rx.recv().await {
// 토렌트 레벨 제한
self.ratelimits
.prepare_for_upload(NonZeroU32::new(ci.size).unwrap())
.await?;
// 세션 레벨 제한
if let Some(session) = self.shared.session.upgrade() {
session.ratelimits
.prepare_for_upload(NonZeroU32::new(ci.size).unwrap())
.await?;
}
let _ = tx.send(WriterRequest::ReadChunkRequest(ci));
}
Ok(())
```

```
}
```

계층화된 제어 특징:

- **이중 제한**: 토렌트 레벨과 세션 레벨 제한 동시 적용
- **우선순위**: 더 제한적인 설정이 우선 적용
- **동적 조정**: 런타임에 제한값 변경 가능

2.3 연결 수 제한 및 관리

```
// 토렌트 상태에서의 피어 관리
pub struct TorrentStateLive {
// 피어 연결 상태 추적
peers: Arc<DashMap<SocketAddr, PeerState>>,
// ...
}

// 세션 옵션에서의 연결 제한
#[derive(Default)]
pub struct SessionOptions {
pub concurrent_init_limit: Option<usize>, // 동시 초기화 수
// ...
}
```

3. 우선순위 기반 리소스 할당

3.1 파일 우선순위 시스템

```
// 파일 우선순위 타입 정의
pub(crate) type FilePriorities = Vec<usize>;


// 토렌트 옵션에서의 파일 선택
#[derive(Default, Serialize, Deserialize)]
pub struct AddTorrentOptions {
pub only_files_regex: Option<String>,
pub only_files: Option<Vec<usize>>,
// ...
}


// 우선순위 기반 파일 처리
fn compute_only_files(
info: &ValidatedTorrentMetaV1Info<ByteBufOwned>,
only_files: Option<Vec<usize>>,
only_files_regex: Option<String>,
list_only: bool,
) -> anyhow::Result<Option<Vec<usize>>> {
match (only_files, only_files_regex) {
(Some(only_files), None) => {
// 명시적 파일 ID 목록 검증
```

```
let total_files = info.iter_file_lengths().count();

for id in only_files.iter().copied() {

if id >= total_files {

bail!("file id {} is out of range", id);

}

}

Ok(Some(only_files))

}

(None, Some(filename_re)) => {

// 정규식 기반 파일 선택

let only_files = compute_only_files_regex(info, &filename_re)?;

Ok(Some(only_files))

}

(None, None) => Ok(None),

_ => bail!("only_files and only_files_regex are mutually exclusive"),

}

}
```

우선순위 할당 특징:

- **선택적 다운로드**: 특정 파일만 다운로드하여 리소스 절약
- **정규식 지원**: 패턴 기반 파일 선택
- **검증**: 파일 ID 유효성 검사

3.2 청크 우선순위 관리

```rust
// 청크 트래커에서의 우선순위 처리
pub struct ChunkTracker {
// 청크 상태 및 우선순위 관리
// ...
}


// 피어 청크 요청자에서의 우선순위 적용
impl PeerHandler {
async fn task_peer_chunk_requester(&self) -> crate::Result<()> {
// 우선순위 기반 청크 요청 로직
// ...
}
}
```

4. QoS 모니터링 및 통계 시스템

4.1 실시간 통계 수집

```rust
// 세션 통계 구조체
pub struct SessionStats {
pub counters: SessionCountersAtomic,
pub peers: Arc<AggregatePeerStatsAtomic>,
pub down_speed_estimator: SpeedEstimator,
```

```
pub up_speed_estimator: SpeedEstimator,
pub startup_time: Instant,
}

// 원자적 카운터 정의 (매크로 생성)
gen_stats!(SessionCountersAtomic SessionCountersSnapshot, [
fetched_bytes u64,
uploaded_bytes u64,
blocked_incoming u64,
blocked_outgoing u64
], []);
```

통계 수집 특징:

- **원자적 카운터**: 멀티스레드 환경에서 안전한 통계 수집
- **속도 추정**: 5초 윈도우 기반 속도 계산
- **피어 통계**: 연결 상태별 피어 수 추적

4.2 속도 추정 및 모니터링

```
impl Session {
pub(crate) fn start_speed_estimator_updater(self: &Arc<Self>) {
self.spawn(
debug_span!(parent: self.rs(), "speed_estimator"),
"speed_estimator",
```

```
{
let s = Arc::downgrade(self);
async move {
let mut i = tokio::time::interval(Duration::from_secs(1));
loop {
i.tick().await;
let s = s.upgrade().context("session is dead")?;
let now = Instant::now();
let fetched = s.stats.counters.fetched_bytes.load(Ordering::Relaxed);
let uploaded = s.stats.counters.uploaded_bytes.load(Ordering::Relaxed);
// 속도 추정기 업데이트
s.stats.down_speed_estimator.add_snapshot(fetched, None, now);
s.stats.up_speed_estimator.add_snapshot(uploaded, None, now);
}
}
},
)
}
}
```

모니터링 특징:

- **주기적 업데이트**: 1초 간격으로 통계 업데이트
- **약한 참조**: 메모리 누수 방지를 위한 약한 참조 사용

- **실시간 계산**: 실시간 속도 및 처리량 계산

4.3 Prometheus 메트릭 지원

```
impl SessionStatsSnapshot {
pub fn as_prometheus(&self, mut out: &mut String) {
use core::fmt::Write;
macro_rules! m {
($type:ident, $name:ident, $value:expr) => {{
writeln!(&mut out, concat!("# TYPE ", stringify!($name), " ", stringify!($type))
writeln!(&mut out, concat!(stringify!($name), " {}"), $value).unwrap();
}};
}

m!(counter, rqbit_fetched_bytes, self.counters.fetched_bytes);
m!(counter, rqbit_uploaded_bytes, self.counters.uploaded_bytes);
m!(counter, rqbit_blocked_incoming, self.counters.blocked_incoming);
m!(counter, rqbit_blocked_outgoing, self.counters.blocked_outgoing);
m!(gauge, rqbit_download_speed_bytes, self.download_speed.as_bytes());
m!(gauge, rqbit_upload_speed_bytes, self.upload_speed.as_bytes());
m!(gauge, rqbit_uptime_seconds, self.uptime_seconds);
// 피어 상태별 메트릭
writeln!(&mut out, "# TYPE rqbit_peers_live gauge").unwrap();
writeln!(&mut out, "rqbit_peers_live{{kind=\"tcp\"}} {}", self.peers.live_tcp).u
```

```
writeln!(&mut out, "rqbit_peers_live{{kind=\"utp\"}} {}", self.peers.live_utp).u
writeln!(&mut out, "rqbit_peers_live{{kind=\"socks\"}} {}", self.peers.live_sock
}
}
```

메트릭 특징:

- **표준 형식**: Prometheus 표준 메트릭 형식 지원
- **다양한 타입**: Counter, Gauge 타입 메트릭
- **레이블 지원**: 연결 타입별 세분화된 메트릭

5. 리소스 제한 통합 아키텍처

5.1 다층 제어 시스템

```
세션 레벨 제한
├── 전역 대역폭 제한
├── 동시 초기화 수 제한
└── 전역 연결 수 제한

토렌트 레벨 제한
├── 토렌트별 대역폭 제한
├── 파일 우선순위
└── 피어 연결 수 제한
```

```
피어 레벨 제한
├── 개별 피어 요청 수 제한
├── 연결 타임아웃
└── Keep-alive 간격
```

5.2 백프레셔 전파 메커니즘

1. **메모리 압박**: 디스크 쓰기 큐 포화 → 네트워크 읽기 중단
2. **대역폭 제한**: 레이트 리미터 → 비동기 대기 → 자연스러운 속도 조절
3. **연결 제한**: 세마포어 → 새 연결 대기 → 리소스 보호

5.3 동적 조정 능력

- **런타임 설정 변경**: `ArcSwapOption` 을 통한 무중단 설정 변경
- **상황 적응**: 네트워크 상태에 따른 자동 조정
- **사용자 제어**: API를 통한 실시간 제한값 조정

6. 성능 및 안정성 영향

6.1 시스템 보호 효과

- **메모리 보호**: OOM 방지를 위한 메모리 사용량 제한
- **CPU 보호**: 동시 작업 수 제한을 통한 CPU 과부하 방지
- **네트워크 보호**: 대역폭 제한을 통한 네트워크 혼잡 방지

6.2 QoS 보장

- **공정성**: 토렌트 간 리소스 공정 분배
- **우선순위**: 중요한 파일 우선 다운로드
- **안정성**: 과부하 상황에서도 안정적 동작

6.3 확장성

- **수평 확장**: 다중 토렌트 동시 처리
- **수직 확장**: 시스템 리소스에 따른 자동 조정
- **탄력성**: 부하 변화에 따른 동적 적응

이러한 포괄적인 리소스 제한 및 QoS 시스템은 rqbit이 다양한 환경에서 안정적이고 효율적으로 동작할 수 있게 하는 핵심 요소입니다.